Chapter 13

RL for Large Reasoning Models

The emergence of large reasoning models represents one of the most significant developments in modern AI. Unlike standard language model training, which optimizes for next-token prediction, reasoning-focused RL teaches models to *think before answering*—allocating additional computation at inference time to explore, verify, and refine intermediate steps. This section provides a comprehensive technical treatment of the methods, architectures, and scaling laws that underpin this paradigm.

13.1 Motivation and Background

13.1.1 Why Reasoning Requires Different RL Approaches

Standard RLHF (Section 4.3) optimizes a single scalar reward over a complete response. For tasks requiring multi-step reasoning—mathematics, formal verification, competitive programming, scientific derivation—this formulation is insufficient for several reasons:

- **Sparse rewards**: A math problem may require 20 intermediate steps; a single outcome reward provides no gradient signal for the intermediate steps that led to an error.
- **Long horizons**: Reasoning chains can span hundreds to thousands of tokens, creating severe credit assignment problems.
- **Combinatorial search**: The space of valid reasoning paths is exponentially large; the model must learn to search this space efficiently.
- **Verifiability**: Unlike subjective text quality, mathematical and logical correctness is objectively verifiable, enabling automated reward computation without human annotation.

Key Insight: Reasoning as a Search Problem

Multi-step reasoning can be framed as a **search problem** over a tree of partial solutions. Each node in the tree is a reasoning state (prefix of the chain-of-thought), each edge is a reasoning step (a token or sentence), and the leaves are final answers. RL for reasoning teaches the model to navigate this tree efficiently—exploring promising branches, backtracking from dead ends, and allocating compute where it matters most.

13.1.2 Chain-of-Thought: Emergent Behavior vs. Trained Capability

Chain-of-thought (CoT) reasoning was first observed as an *emergent* capability in sufficiently large language models [374]: when prompted with step-by-step examples, large models (typically $\geq$100B parameters) spontaneously produced intermediate reasoning steps that improved accuracy. This raised a fundamental question: is CoT an emergent property of scale, or can it be explicitly trained?

The answer, as demonstrated by DeepSeek-R1 and related work, is **both**—but with important nuances:

- **Emergent CoT** arises from in-context learning and requires large base models. It is brittle, prompt-sensitive, and does not generalize robustly.

- **Trained CoT** via RL produces models that *intrinsically* generate reasoning chains as part of their generation process, independent of prompting style. These chains are longer, more exploratory, and exhibit qualitatively different behaviors (self-correction, backtracking, verification).

The "Aha Moment" Phenomenon (DeepSeek-AI et al. 2025)

During RL training of reasoning models, researchers at DeepSeek observed a striking emergent behavior: at a certain point in training, models spontaneously began to *reconsider* their initial approaches mid-chain, using phrases like "Wait, let me reconsider..." or "Actually, I think I made an error...". This self-correction behavior—which was *not* explicitly trained—emerged purely from the RL objective of maximizing final-answer accuracy. It suggests that RL can discover meta-cognitive strategies that are instrumentally useful for solving hard problems.

13.1.3 Test-Time Compute Scaling Laws

A central empirical finding motivating reasoning model research is that **test-time compute scales predictably with performance**. Let C_{train} denote training compute (FLOPs) and C_{test} denote inference compute (tokens generated). The key observation is:

$$\text{Accuracy}(C_{\text{train}}, C_{\text{test}}) \approx f(\alpha \log C_{\text{train}} + \beta \log C_{\text{test}}) \tag{13.1}$$

for some monotone function f and constants $\alpha, \beta > 0$. This implies that a smaller model with more inference compute can match a larger model with less inference compute—a fundamental shift in the compute-performance tradeoff.

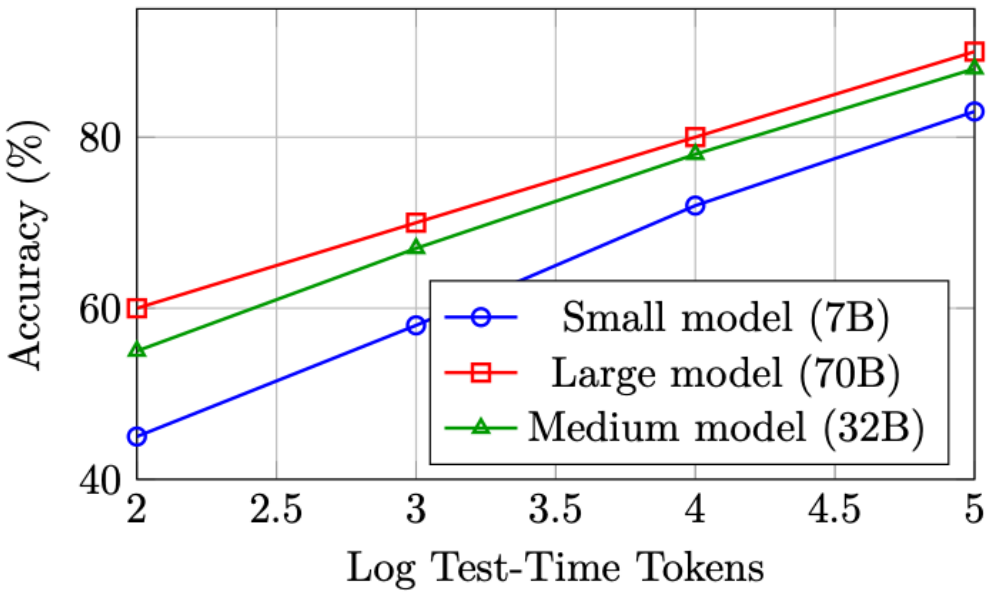

Figure 13.1: Schematic test-time compute scaling curves. Performance improves log-linearly with inference tokens across model sizes, and smaller models with more compute can approach larger models with less compute.

The practical implication is profound: **reasoning models trade training compute for inference compute**. Rather than always deploying the largest possible model, one can deploy a smaller, reasoning-capable model and allocate more tokens to "thinking" on hard problems.

13.2 Test-Time Scaling Methods

The scaling laws above show that investing more compute at inference can dramatically improve reasoning performance. This section provides a comprehensive treatment of the **methods** that operationalize test-time scaling — from simple chain-of-thought to sophisticated tree and graph search algorithms. These methods form a spectrum trading inference cost for accuracy, and understanding their structure is essential for designing modern reasoning systems.

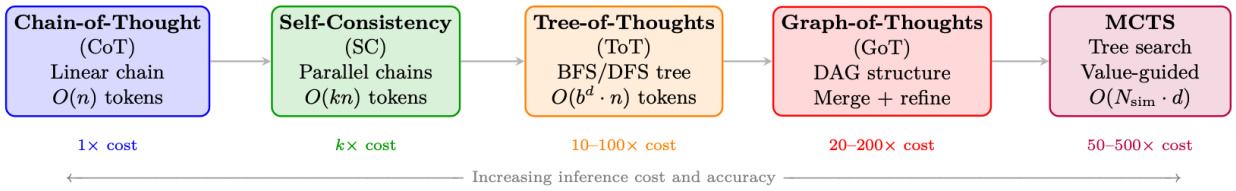

Figure 13.2: Spectrum of test-time scaling methods. Each method trades additional inference compute for improved reasoning accuracy. Methods build on each other conceptually: CoT introduces explicit reasoning, Self-Consistency adds sampling, ToT adds structured search, GoT adds merging operations, and MCTS adds learned value guidance.

13.2.1 Chain-of-Thought (CoT)

Chain-of-Thought prompting [374] is the foundation of all test-time scaling methods. Rather than directly outputting an answer, the model generates intermediate reasoning steps that decompose complex problems into manageable sub-problems.

Zero-Shot CoT. Kojima et al. [189] demonstrated that appending "Let's think step by step" to a prompt elicits reasoning behavior without any exemplars. This simple trigger activates latent reasoning capabilities in sufficiently large models ($\geq$100B parameters).

Few-Shot CoT. Wei et al. [374] showed that providing a few exemplars with explicit reasoning traces enables smaller models to reason effectively:

$$\text{Prompt} = [(x_1, z_1, y_1), (x_2, z_2, y_2), \ldots, (x_k, z_k, y_k), (x_{\text{test}}, ?)] \tag{13.2}$$

where z_i are hand-written reasoning traces for exemplar (x_i, y_i).

Formal characterization. CoT converts a single-step prediction $p(y|x)$ into a multi-step sequential generation:

$$p(y|x) = \sum_z p(y|x, z) \cdot p(z|x) \approx p(y|x, z^*) \cdot p(z^*|x) \tag{13.3}$$

where $z^* = (z_1, z_2, \ldots, z_T)$ is the greedy reasoning chain. The summation over all possible chains is intractable; standard CoT uses a single sample (greedy or temperature sampling).

Limitations. Single-chain CoT is fragile: if an early reasoning step is wrong, all subsequent steps build on a flawed foundation with no mechanism for recovery.

13.2.2 Self-Consistency (Majority Voting)

Self-Consistency [367] addresses CoT's single-chain fragility by sampling **multiple independent reasoning chains** and taking a majority vote over the final answers:

$$\hat{y} = \arg\max_y \sum_{i=1}^{N} \mathbf{1}[y_i = y], \quad \text{where } (z_i, y_i) \sim p(\cdot|x),\ T > 0 \tag{13.4}$$

Key properties:

- Uses temperature $T > 0$ sampling to generate diverse chains (typically $T = 0.7$–1.0)
- No interaction between chains — fully parallelizable
- Accuracy improves monotonically with N (diminishing returns after $N \approx 40$)
- On GSM8K: CoT = 56.5%, Self-Consistency (N=40) = 74.4% (with PaLM-540B [55])
- Equivalent to **Best-of-N with outcome reward** (majority vote acts as implicit ORM)

Why Majority Voting Works

If the model has probability $p > 0.5$ of generating a correct reasoning chain, then by the law of large numbers, majority voting over N independent samples approaches 100% accuracy as $N \to \infty$. Even with $p = 0.3$ (model is usually wrong), if correct answers concentrate on one value while incorrect answers are diverse, majority voting still recovers the correct answer. This is the statistical foundation of test-time scaling.

13.2.3 Tree-of-Thoughts (ToT)

Tree-of-Thoughts [409] generalizes CoT from a **linear chain** to a **tree structure**, enabling the model to explore multiple reasoning paths, evaluate intermediate states, and backtrack from unpromising branches. This introduces deliberate planning into the reasoning process.

Core Abstraction. A reasoning problem is decomposed into a search over a tree where:

- **Root**: Initial problem statement x
- **Nodes**: Partial reasoning states $s = (x, z_1, \ldots, z_k)$
- **Edges**: Individual reasoning steps ("thoughts") z_{k+1}
- **Leaves**: Complete solutions with final answers
- **Value function**: $V(s)$ estimates how promising a partial solution is

Formal Definition.

$$\text{ToT} = (\mathcal{G}, \mathcal{E}, V, \pi_\theta, \text{Search}) \tag{13.5}$$

where:

- $\mathcal{G}$: **Thought generator** — produces b candidate next thoughts: $\{z^{(1)}, \ldots, z^{(b)}\} \sim \pi_\theta(\cdot|s)$
- $\mathcal{E}$: **State evaluator** — scores partial solutions: $V(s) \in \{\textit{sure}, \textit{maybe}, \textit{impossible}\}$ or $V(s) \in [0, 1]$
- π_θ: The language model generating thoughts
- Search: Search algorithm (BFS or DFS)

Search Algorithms. BFS (Breadth-First Search):

1. Generate b candidate thoughts for each node at current depth
2. Evaluate all candidates with $V(\cdot)$
3. Keep top-k most promising states (beam search)
4. Advance all k states to the next level

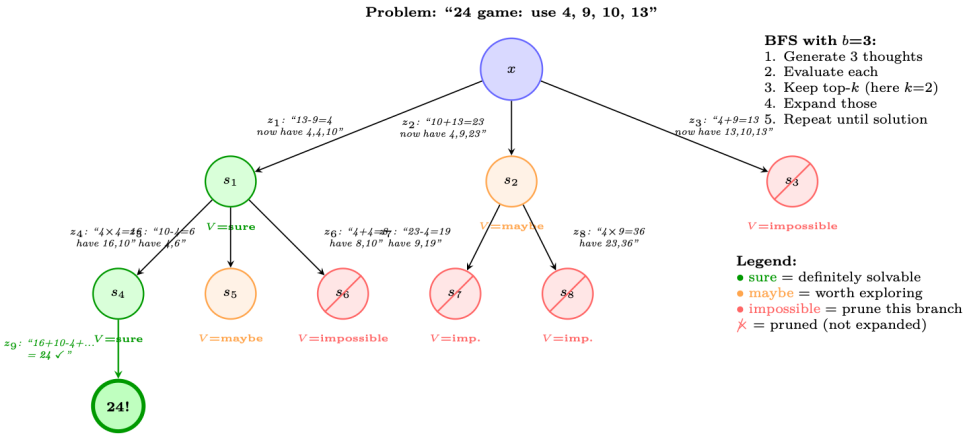

Figure 13.3: Tree-of-Thoughts on the "Game of 24" task: use operations on 4, 9, 10, 13 to make 24. At each level, the model generates $b = 3$ candidate thoughts, evaluates each (sure/maybe/impossible), prunes unpromising branches, and expands the most promising ones. The green path leads to a solution; red paths are pruned early.

5. Repeat until a solution is found or depth limit reached

DFS (Depth-First Search):

1. Generate b candidate thoughts for current state
2. Evaluate: if $V(s) = \textit{impossible}$, backtrack immediately
3. If $V(s) = \textit{sure/maybe}$, recurse deeper (pick the most promising)
4. If depth limit reached without solution, backtrack
5. Continue until solution found or all branches explored

ToT: Value Evaluation Prompt

```
# The LLM evaluates partial reasoning states:
EVAL_PROMPT = """Evaluate if this partial solution can reach 24.

Numbers remaining: [4, 4, 10]
Steps so far: 13 - 9 = 4

Can these remaining numbers (4, 4, 10) be combined using +,-,*,/
to make 24?

Analysis: 4 * (10 - 4) = 4 * 6 = 24. Yes!

Judge: sure/maybe/impossible
Answer: sure"""

# Thought generation prompt:
GEN_PROMPT = """Input: 4 9 10 13
Possible next steps:
1. 13 - 9 = 4 (left: 4 4 10)
2. 10 + 13 = 23 (left: 4 9 23)
3. 9 - 4 = 5 (left: 5 10 13)
..."""
```

Computational Cost. For ToT with branching factor b, depth d, and beam width k:

$$\text{LLM calls (BFS)} = \underbrace{k \cdot b}_{\text{generation}} + \underbrace{k \cdot b}_{\text{evaluation}} = 2kb \text{ per level} \implies \text{Total} = 2kbd \tag{13.6}$$

For the 24 game: $b = 3, k = 2, d = 3 \implies 36$ LLM calls vs. 1 for standard CoT.

Results. On the Game of 24 (a challenging arithmetic reasoning task), ToT achieves 74% success rate vs. CoT's 4% — a massive improvement from structured search over the same base model (GPT-4).

13.2.4 Graph-of-Thoughts (GoT)

Graph-of-Thoughts [25] extends ToT from a tree to a **directed acyclic graph (DAG)**, introducing a critical capability: **merging** partial solutions from different branches. This allows the model to synthesize insights from multiple reasoning paths into a single refined solution.

Key Operations. GoT introduces three operations beyond ToT:

- **Generate**: Produce new thoughts from a state (same as ToT)
- **Aggregate/Merge**: Combine multiple thoughts into one refined thought — this is impossible in a tree
- **Refine**: Iteratively improve a thought based on feedback
- **Score**: Evaluate thought quality (same as ToT's value function)

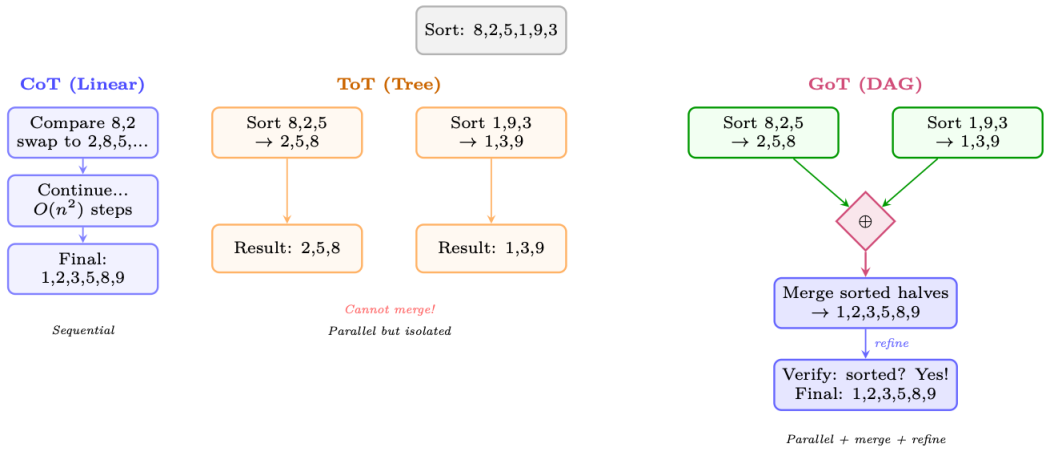

Figure 13.4: Comparison of CoT (linear chain), ToT (tree — branches but no merging), and GoT (DAG — branches can merge). For a sorting task, GoT can split the array into sub-problems, solve them independently (parallel), then **merge** the results — impossible in a pure tree structure. This enables divide-and-conquer reasoning.

Graph Operations (formal). Let $\mathcal{V} = \{v_1, \ldots, v_n\}$ be thought vertices and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ be directed edges. GoT supports:

$$\textbf{Generate}(v) : v \to \{v_{c_1}, \ldots, v_{c_b}\} \quad \text{(create children)} \tag{13.7}$$

$$\textbf{Aggregate}(v_1, \ldots, v_k) \to v_{\text{merged}} \quad \text{(merge } k \text{ thoughts into one)} \tag{13.8}$$

$$\textbf{Refine}(v, n) \to v' \quad \text{(improve } v \text{ through } n \text{ iterations)} \tag{13.9}$$

$$\textbf{Score}(v) \to s \in [0, 1] \quad \text{(evaluate thought quality)} \tag{13.10}$$

The **Aggregate** operation is the key differentiator: it creates edges from multiple parent nodes to a single child, forming a DAG rather than a tree. This enables:

- **Divide-and-conquer**: Split problem → solve sub-problems in parallel → merge solutions
- **Ensemble reasoning**: Generate multiple perspectives, then synthesize the best ideas
- **Iterative refinement**: Feed evaluation results back to improve earlier thoughts

Results. On sorting (a task requiring merging), GoT achieves 62% cost reduction vs. ToT at equivalent quality. On set intersection and keyword counting, GoT matches ToT quality with 30–40% fewer LLM calls due to the merge operation enabling more efficient decomposition.

13.2.5 Best-of-N with Reward Models

Best-of-N (BoN) [262, 339] is the simplest scaling method that uses a **learned reward model** to select among candidates:

$$y^* = \arg\max_{y \in \{y_1, \ldots, y_N\}} R_\phi(x, y), \quad y_i \sim \pi_\theta(\cdot|x) \tag{13.11}$$

Variants by reward model type:

- **BoN with ORM**: Score complete solutions; select the highest-scoring one. Equivalent to Self-Consistency when ORM $\approx$ correctness check.
- **BoN with PRM**: Score at each reasoning step; select the solution with highest minimum step score (least likely to have an error at any step).
- **Weighted BoN**: Weight candidates by reward: $y^* \sim \text{softmax}(R(y_1)/\tau, \ldots, R(y_N)/\tau)$.

BoN Scaling Law

For a model with per-sample accuracy p, the probability of at least one correct sample in N tries:

$$P(\text{success with BoN}) = 1 - (1-p)^N \tag{13.12}$$

With a perfect reward model (oracle that always selects correctly):

- $p = 0.3, N = 10$: success $= 97\%$
- $p = 0.1, N = 50$: success $= 99.5\%$

In practice, imperfect reward models cap the effective N — beyond $N \approx 64$–256, reward model errors dominate and accuracy plateaus or decreases (**reward hacking**).

13.2.6 Monte Carlo Tree Search (MCTS) for Reasoning

MCTS [187, 331] combines the structured exploration of ToT with **learned value estimates** and **visit-count statistics** to allocate inference compute optimally. Originally developed for game-playing (AlphaGo [331]), MCTS has been adapted for LLM reasoning by systems including AlphaProof [68] and rStar [297].

Algorithm (adapted for LLM reasoning). Each MCTS iteration consists of four phases:

UCB for Reasoning. Node selection uses PUCT (Predictor + UCB applied to Trees):

$$a^* = \arg\max_a \left[Q(s,a) + c_{\text{puct}} \cdot P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)} \right] \tag{13.13}$$

where $P(s,a) = \pi_\theta(a|s)$ is the LLM's prior probability of generating step a from state s. This biases exploration toward steps the LLM already considers likely, while the UCB term encourages trying under-explored alternatives.

MCTS for Math Reasoning: Running Example

Problem: Prove that $\sqrt{2}$ is irrational.
Iteration 1 (Selection $\rightarrow$ root, Expansion):

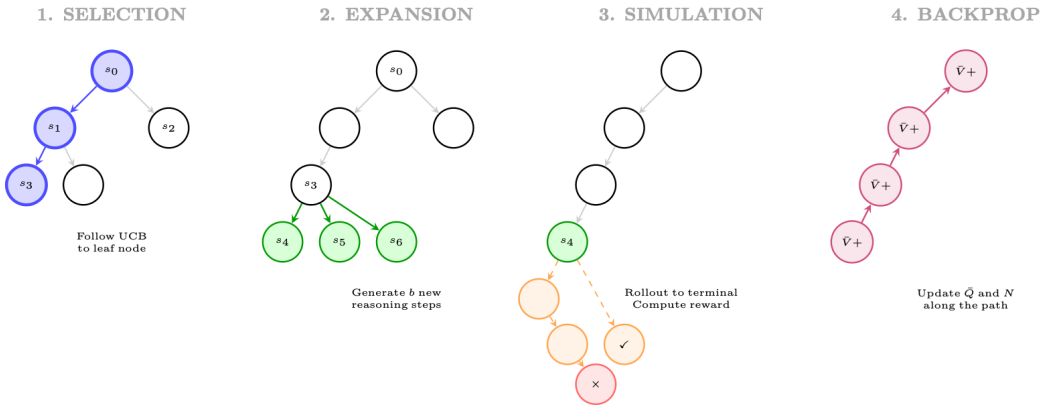

Figure 13.5: Four phases of MCTS for reasoning: (1) **Selection**: traverse tree using UCB to find a promising leaf; (2) **Expansion**: generate new reasoning steps from the leaf; (3) **Simulation**: complete the reasoning to a terminal state and evaluate; (4) **Backpropagation**: update value estimates along the path.

- Generate 3 candidate first steps:
  1. "Assume for contradiction that $\sqrt{2} = p/q$ in lowest terms." ($P = 0.7$)
  2. "Consider the decimal expansion of $\sqrt{2} = 1.414...$" ($P = 0.15$)
  3. "Use the fundamental theorem of arithmetic." ($P = 0.10$)
- Rollout from z_1: reaches correct proof in 4 steps $\rightarrow r = 1.0$
- Rollout from z_2: fails (decimal doesn't prove irrationality) $\rightarrow r = 0.0$
- Backprop: $Q(s_0, z_1) = 1.0$, $N(s_0, z_1) = 1$

Iteration 2 (Selection: pick z_1 by UCB):

- Expand from state "Assume $\sqrt{2} = p/q$...":
  1. "Then $2 = p^2/q^2$, so $p^2 = 2q^2$." ($P = 0.8$)
  2. "Then p and q share no common factors." ($P = 0.15$)
- Rollout from z_4: correct continuation $\rightarrow r = 1.0$
- Backprop: $Q(s_0, z_1) = 1.0$, $Q(s_1, z_4) = 1.0$

After 20 iterations: The tree has explored 8 distinct reasoning paths. The most-visited path is selected as the final proof: $z_1 \rightarrow z_4 \rightarrow z_6 \rightarrow z_8$ (classical proof by contradiction via even/odd argument).

Comparison: ToT vs. MCTS.

13.2.7 Beam Search over Reasoning Steps

Beam search — long standard in NMT and text generation — can be applied at the *reasoning step* level rather than the token level. Instead of tracking the top-k token sequences, we track the top-k **reasoning prefixes**:

$$\mathcal{B}_d = \text{top-}k \left\{ (s_1, \ldots, s_d) : \sum_{i=1}^{d} \log \pi_\theta(s_i | s_{<i}) + \lambda \cdot V_\phi(s_1, \ldots, s_d) \right\} \tag{13.14}$$

where the scoring combines the LLM log-probability (fluency) with a value model estimate (correctness). This is effectively ToT-BFS with a learned value function rather than a prompted one.

Table 13.1: Tree-of-Thoughts vs. Monte Carlo Tree Search for reasoning.

Dimension	ToT	MCTS
Value estimation	LLM prompt ("sure/maybe/impossible")	Learned value network + rollout statistics
Exploration	Fixed beam width; no revisiting	UCB adaptively allocates budget to promising nodes
Compute allocation	Uniform across depth levels	Focused: more simulations on harder subproblems
Training integration	No training; pure prompting	Can distill MCTS policy into the base model [331]
Best for	Simple branching problems (24 game)	Complex problems requiring deep exploration (proofs, code)

13.2.8 Iterative Refinement and Self-Correction

Rather than exploring *breadth* (multiple parallel chains), iterative refinement invests compute in *depth* — repeatedly improving a single solution:

$$y^{(t+1)} = \text{LLM}\Big(\text{"Improve this solution:"}, y^{(t)}, \text{"Errors found:"}, e^{(t)}\Big) \tag{13.15}$$

where $e^{(t)}$ may come from:

- **Self-verification**: Ask the model to check its own answer
- **External verification**: Run code, check math symbolically
- **Critic model**: A separate model identifies errors

Notable methods: **Self-Refine** [246] (iterative self-feedback), **Reflexion** [326] (verbal RL via reflections stored in memory), and **LATS** [438] (tree search + reflection-based pruning).

13.2.9 Method Comparison and Selection Guide

Table 13.2: Comprehensive comparison of test-time scaling methods.

Method	Structure	LLM Calls	Parallelizable	Needs RM?	Best For
CoT [374]	Chain	1	N/A	No	Easy–medium problems
Self-Consistency [367]	Parallel chains	N	✓ Fully	No (majority vote)	Math with discrete answers
Best-of-N + ORM	Parallel chains	$N + 1$	✓ Fully	Yes (ORM)	General tasks with good RM
Best-of-N + PRM	Parallel chains	$N + N{\cdot}K$	✓ Fully	Yes (PRM)	Complex multi-step reasoning
ToT [409]	Tree (BFS/DFS)	$O(kbd)$	Partial	LLM-as-judge	Structured search problems
GoT [25]	DAG	$O(kbd)$	Partial	LLM-as-judge	Decomposable problems
MCTS [187]	Tree + values	$O(N_{\text{sim}} \cdot d)$	Partial	Yes (value net)	Hard proofs, coding
Self-Refine [246]	Linear (iterative)	$2T$	No	Self-critic	Open-ended generation
LATS [438]	Tree + reflection	$O(N \cdot d)$	Partial	LLM-as-judge	Agent tasks

When to Use Which Method

- **Budget $<$ 5× base cost**: Use CoT or Self-Consistency. Maximum bang for the buck.
- **Budget 5–50×**: Use Best-of-N with PRM (if you have a good reward model) or ToT-BFS with $b = 3, k = 2$.
- **Budget 50–500×**: Use MCTS with a trained value function. This is the regime where DeepSeek-R1 and OpenAI o1 operate — long reasoning chains with implicit tree search.
- **Parallelism required**: Self-Consistency and Best-of-N are fully parallel; ToT/MCTS require sequential depth expansion.
- **No reward model available**: Use Self-Consistency (majority vote) or ToT with LLM-as-

judge evaluation.

- **Decomposable problems**: GoT excels when the problem has natural sub-problems (sorting, multi-document synthesis, code with modules).

The Implicit Test-Time Scaling in Reasoning Models

Modern reasoning models (DeepSeek-R1 [72], OpenAI o1/o3 [275, 278]) perform **implicit test-time scaling** via long chain-of-thought generation. Their "thinking" tokens serve a function analogous to MCTS rollouts: the model explores multiple approaches, backtracks ("Wait, let me reconsider..."), verifies intermediate steps, and allocates more tokens to harder sub-problems. The key insight of R1/o1 training is that GRPO/RL teaches the model to perform this implicit search *within a single generation*, eliminating the need for external orchestration (ToT prompts, MCTS infrastructure). The model becomes its own search algorithm.

13.3 DeepSeek-R1

DeepSeek-R1 [72] is the first fully open-source large reasoning model to match or exceed OpenAI o1 on major benchmarks. Its training pipeline is technically transparent and has become the de facto reference implementation for RL-based reasoning.

13.3.1 Two-Stage Training Pipeline

Stage 1: Cold-Start Supervised Fine-Tuning The base model (DeepSeek-V3) is first fine-tuned on a small, carefully curated dataset of long chain-of-thought examples. This "cold start" phase serves two purposes:

1. **Format initialization**: The model learns to produce reasoning in the `<think>...</think>` format before emitting a final answer.
2. **Stability**: Without cold-start SFT, pure RL from scratch on the base model produces unstable training dynamics and degenerate outputs (e.g., language mixing, repetitive loops).

The cold-start dataset contains only ∼thousands of examples, deliberately kept small to avoid over-constraining the reasoning style that RL will later discover.

Stage 2: GRPO-Based Reinforcement Learning After cold-start SFT, the model undergoes large-scale RL using Group Relative Policy Optimization (GRPO). The full GRPO objective as used in R1 is described in Section 13.3.3.

R1 Training Pipeline Summary

1. **Base model**: DeepSeek-V3 (671B MoE, 37B active parameters)
2. **Cold-start SFT**: ∼thousands of long-CoT examples, format: `<think>...</think><answer>...</answer>`
3. **RL phase**: GRPO with verifiable rewards on math + code problems
4. **Rejection sampling**: Generate multiple solutions, keep correct ones
5. **SFT on RL outputs**: Fine-tune on high-quality RL-generated chains
6. **Final RL**: Second RL phase for alignment + helpfulness

13.3.2 Reward Design: Accuracy and Format Rewards

A key design choice in R1 is the **absence of a process reward model**. Instead, R1 uses two simple, automatically computable rewards:

Accuracy Reward For math problems with verifiable answers:

$$r_{\text{acc}}(y, y^*) = \begin{cases} 1 & \text{if } \texttt{verify}(y, y^*) = \texttt{True} \\ 0 & \text{otherwise} \end{cases} \tag{13.16}$$

where y is the model's final answer (extracted from `<answer>` tags) and y^* is the ground-truth answer. The `verify` function uses symbolic math comparison (e.g., SymPy) to handle equivalent forms.

For code problems, the accuracy reward is determined by passing test cases:

$$r_{\text{acc}}^{\text{code}}(y, \mathcal{T}) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \mathbf{1}[\texttt{execute}(y, t) = \texttt{expected}(t)] \tag{13.17}$$

Format Reward To enforce the `<think>...</think>` structure:

$$r_{\text{fmt}}(y) = \begin{cases} 1 & y \text{ has valid } \langle\text{think}\rangle \text{ and } \langle\text{answer}\rangle \text{ tags} \\ 0 & \text{otherwise} \end{cases} \tag{13.18}$$

Combined Reward

$$r(y, y^*) = r_{\text{acc}}(y, y^*) + \lambda_{\text{fmt}} \cdot r_{\text{fmt}}(y) \tag{13.19}$$

with $\lambda_{\text{fmt}} = 0.1$ in the original implementation (small enough not to dominate, large enough to prevent format collapse).

No Process Reward Model

A notable and surprising finding of R1 is that **no process reward model (PRM) is needed**. Despite the long reasoning chains, outcome-only rewards are sufficient for RL to discover high-quality reasoning strategies. The authors hypothesize that the verifiable nature of math/code rewards provides sufficient signal, and that PRMs introduce their own failure modes (reward hacking at the step level). This contrasts with the approach taken by OpenAI (Section 13.4).

13.3.3 GRPO Formulation for R1

GRPO [323] is a policy gradient method that avoids training a separate value network by estimating advantages from a *group* of sampled responses. For a question q, GRPO samples G responses $\{y_1, y_2, \ldots, y_G\}$ from the current policy π_θ and computes advantages relative to the group mean.

Group Sampling and Advantage Normalization Given question q, sample G outputs:

$$\{y_i\}_{i=1}^{G} \sim \pi_\theta(\cdot \mid q) \tag{13.20}$$

Compute rewards $\{r_i\}_{i=1}^{G}$ using the reward function from Eq. [eq:r1_combined_reward]. The normalized advantage for response i is:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon} \tag{13.21}$$

where $\mu_r = \frac{1}{G}\sum_{i=1}^{G} r_i$, $\sigma_r = \sqrt{\frac{1}{G}\sum_{i=1}^{G}(r_i - \mu_r)^2}$, and $\epsilon = 10^{-8}$ for numerical stability.

GRPO Objective The GRPO objective clips the probability ratio (as in PPO) and adds a KL penalty against a reference policy π_{ref}:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_{q\sim\mathcal{D},\,\{y_i\}\sim\pi_\theta(\cdot|q)}\Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|} \min\Big(\rho_{i,t}\,\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\,\hat{A}_i\Big) - \beta\,\mathbb{D}_{\text{KL}}[\pi_\theta \,\|\, \pi_{\text{ref}}]\Bigg] \tag{13.22}$$

where:

- $\rho_{i,t} = \dfrac{\pi_\theta(y_{i,t} \mid q, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid q, y_{i,<t})}$ is the per-token probability ratio
- $\varepsilon \in \{0.1, 0.2\}$ is the PPO clipping parameter
- $\beta > 0$ controls the KL penalty strength
- $|y_i|$ is the length of response i (length normalization prevents bias toward short responses)

KL Penalty Formulation The KL divergence term is computed token-by-token:

$$\mathbb{D}_{\text{KL}}[\pi_\theta \,\|\, \pi_{\text{ref}}] = \mathbb{E}_{y\sim\pi_\theta(\cdot|q)}\left[\sum_{t=1}^{|y|}\log\frac{\pi_\theta(y_t \mid q, y_{<t})}{\pi_{\text{ref}}(y_t \mid q, y_{<t})}\right] \tag{13.23}$$

In practice, R1 uses an unbiased estimator of the KL that avoids computing π_{ref} at every step by using the approximation:

$$\mathbb{D}_{\text{KL}}[\pi_\theta \,\|\, \pi_{\text{ref}}] \approx \frac{\pi_{\text{ref}}(y_t \mid q, y_{<t})}{\pi_\theta(y_t \mid q, y_{<t})} - \log\frac{\pi_{\text{ref}}(y_t \mid q, y_{<t})}{\pi_\theta(y_t \mid q, y_{<t})} - 1 \tag{13.24}$$

which is always non-negative and equals zero when $\pi_\theta = \pi_{\text{ref}}$.

GRPO in Practice: Group Size and Stability

In R1's training, $G = 8$ responses are sampled per question. This is a critical hyperparameter:

- Too small ($G = 2$): High variance in advantage estimates; training is noisy.
- Too large ($G = 32$): Computational cost scales linearly; diminishing returns.
- $G = 8$: Empirically found to balance variance reduction and compute cost.

The group sampling also provides a natural **curriculum signal**: as training progresses, the model's average reward μ_r increases, and the variance σ_r decreases. Problems where all G responses are correct (or all wrong) contribute zero gradient, naturally focusing learning on problems at the frontier of the model's capability.

13.3.4 Distillation: The R1-Distill Series

A major practical contribution of R1 is demonstrating that **reasoning capabilities can be distilled into much smaller models** via supervised fine-tuning on R1-generated chains. The R1-Distill series (1.5B, 7B, 8B, 14B, 32B, 70B parameters) is trained by:

1. Generating long-CoT solutions to a large problem set using R1 (671B)
2. Filtering to keep only correct solutions
3. Fine-tuning smaller base models (Qwen2.5, Llama-3) on these solutions

Distillation vs. RL for Small Models

A striking finding: **distillation outperforms RL training from scratch on small models**. DeepSeek-R1-Distill-Qwen-7B achieves higher MATH benchmark scores than a 7B model trained with GRPO directly. This suggests that:

- Small models lack the capacity to discover reasoning strategies via RL exploration
- But they *can* learn to imitate reasoning strategies discovered by larger models
- The bottleneck for small models is *exploration*, not *representation*

The distillation approach raises an important question about the nature of reasoning: is the small model truly "reasoning," or is it pattern-matching on the surface form of reasoning chains? Empirically, distilled models show some generalization to novel problem types, suggesting genuine internalization of reasoning strategies rather than pure memorization.

13.4 OpenAI o1/o3 Series

OpenAI's o1 [275] (released September 2024) and subsequent o3/o4-mini [278] models represent the commercial frontier of reasoning model development. While full technical details remain proprietary, the published system cards, technical reports, and empirical observations provide substantial insight into the methodology.

13.4.1 Chain-of-Thought RL with Hidden Reasoning Tokens

The defining architectural choice of o1 is the use of **hidden reasoning tokens**: the model generates an internal chain-of-thought (called a "reasoning trace" or "thinking tokens") that is not shown to the user. Only the final answer is returned. This design has several implications:

- **No format constraints**: The hidden reasoning can use any format, including scratchpad notation, pseudocode, or even non-English reasoning.
- **No reward hacking on style**: Since users never see the reasoning, there is no pressure to make it look "good" rather than be useful.
- **Proprietary protection**: The reasoning process is not exposed, preventing direct imitation.

The training procedure is described as "training models to reason using RL," with the RL objective applied to the complete (hidden reasoning + final answer) sequence, rewarded only on the quality of the final answer.

13.4.2 Process Reward Models vs. Outcome Reward Models

OpenAI's approach is believed to use **Process Reward Models (PRMs)** [215] in addition to outcome rewards, in contrast to DeepSeek-R1's outcome-only approach. This inference is based on OpenAI's published PRM research (PRM800K dataset, "Let's Verify Step by Step") and the o1 system card's description of RL training on reasoning chains, though the exact o1/o3 training recipe has not been publicly disclosed.

Outcome Reward Model (ORM) An ORM scores the complete response (q, y):

$$R_{\text{ORM}}(q, y) \in [0, 1] \tag{13.25}$$

For verifiable tasks (math, code), this reduces to exact-match verification. For open-ended tasks, a learned reward model is used.

Process Reward Model (PRM) A PRM assigns a reward to each reasoning step s_k in the chain $y = (s_1, s_2, \ldots, s_K)$:

$$R_{\text{PRM}}(q, y) = \sum_{k=1}^{K} \gamma^{K-k} \cdot r_k(q, s_1, \ldots, s_k) \tag{13.26}$$

where $r_k \in [0, 1]$ is the step-level reward and $\gamma \in (0, 1]$ is a discount factor. The step-level reward r_k estimates the probability that the partial solution $(s_1, \ldots, s_k)$ leads to a correct final answer:

$$r_k(q, s_1, \ldots, s_k) = P(\text{correct final answer} \mid q, s_1, \ldots, s_k) \tag{13.27}$$

PRM vs. ORM: The Credit Assignment Tradeoff

ORM provides clean, unambiguous rewards but suffers from severe credit assignment problems: a single wrong step early in a 50-step chain receives the same zero reward as a completely random response.
PRM provides dense rewards that directly address credit assignment, but introduces new challenges:

- **Training data**: Step-level labels require human annotation or automated generation (Math-Shepherd, Section 13.6.2).
- **Reward hacking**: Models can learn to produce steps that *look* correct to the PRM without actually being correct.
- **Distribution shift**: PRMs trained on one distribution of reasoning chains may not generalize to the novel chains produced by RL.

The empirical evidence suggests PRMs are beneficial for *search* (selecting among candidate solutions) but their benefit for *training* is less clear.

13.4.3 Inference-Time Compute Scaling

The o1 technical report demonstrates a clear scaling law: **more thinking tokens monotonically improve performance** on hard reasoning tasks. This is operationalized through a "thinking budget" parameter that controls the maximum number of hidden reasoning tokens.

Let T be the thinking token budget. The empirical scaling law observed is approximately:

$$\text{Pass@1}(T) \approx a - b \cdot T^{-c} \tag{13.28}$$

for constants $a, b, c > 0$, where a represents the asymptotic accuracy ceiling and c characterizes the rate of improvement. For AIME 2024, o1 with full thinking budget achieves ~83% accuracy, compared to ~13% for GPT-4o (which uses no extended thinking).

13.4.4 Training Compute vs. Test-Time Compute

A fundamental insight from the o1/o3 series is the **compute equivalence principle**: there exists a tradeoff curve between training compute C_{train} and test-time compute C_{test} such that points on the curve achieve similar performance:

$$\text{Performance}(C_{\text{train}}, C_{\text{test}}) = g(\alpha C_{\text{train}}^{p} + \beta C_{\text{test}}^{q}) \tag{13.29}$$

Empirically, $p \approx q$ for reasoning tasks, suggesting that training and test-time compute are roughly substitutable. This has profound implications for deployment: a smaller, cheaper model with extended thinking can match a larger model on hard problems, at the cost of higher latency.

13.4.5 o3 and o4-mini Architecture Insights

While o3 and o4-mini details remain largely proprietary, several observations have emerged:

- **o3**: Substantially larger thinking budgets than o1; achieves near-human performance on ARC-AGI (87.5% with high compute). Believed to use more sophisticated search strategies during inference.
- **o4-mini**: Demonstrates that *smaller* models with RL-trained reasoning can be highly competitive. Achieves 93% on AIME 2025 with extended thinking, suggesting that model size is less important than reasoning capability for math.
- **Tool use**: o3/o4-mini integrate tool use (code execution, web search) into the reasoning process, allowing the model to verify intermediate steps programmatically.

13.5 QwQ and Qwen Reasoning Models

Alibaba's Qwen team has developed a series of reasoning models (QwQ-32B [351], Qwen3 [352]) that represent the open-source frontier alongside DeepSeek-R1. Their approach differs in several key respects.

13.5.1 Multi-Stage RL Pipeline

The Qwen reasoning pipeline uses a more elaborate multi-stage approach:

1. **Base pretraining**: Qwen2.5 base model with strong mathematical and coding capabilities
2. **SFT on diverse reasoning**: Fine-tuning on a broad mixture of reasoning tasks (math, code, science, logic)
3. **Rejection sampling fine-tuning (RFT)**: Generate N solutions per problem, keep correct ones, fine-tune
4. **RL phase 1**: GRPO on math and code with verifiable rewards
5. **RL phase 2**: Broader RL including instruction following and safety

13.5.2 Rejection Sampling + RL Combination

A key innovation in the Qwen approach is the **iterative combination of rejection sampling and RL**:

1. **Initialize**: Policy π_0 from SFT model.
2. **Rejection Sampling**: Sample N solutions: $\{y_i\}_{i=1}^{N} \sim \pi_{k-1}(\cdot \mid q)$. Keep correct solutions: $\mathcal{Y}^+(q) = \{y_i : r(y_i, y^*) = 1\}$.
3. **SFT update**: $\pi_k^{\text{SFT}} \leftarrow \text{SFT}(\pi_{k-1}, \bigcup_q \mathcal{Y}^+(q))$
4. **RL update**: $\pi_k \leftarrow \text{GRPO}(\pi_k^{\text{SFT}}, \mathcal{D})$
5. **Repeat** steps 2–4 for K iterations to obtain final policy π_K.

The rejection sampling step provides high-quality positive examples that anchor the policy, while RL explores beyond the current distribution. This combination is more stable than pure RL and more capable than pure SFT.

13.5.3 Tool-Integrated Reasoning

QwQ-32B and Qwen3 models support **tool-integrated reasoning**: the model can invoke external tools (Python interpreter, search engine, calculator) during its reasoning chain. This is implemented via special tokens:

```
<think>
Let me solve this step by step.
First, I'll compute the eigenvalues of the matrix.

<tool_call>
{"name": "python", "arguments": {"code": "import numpy as np\nA = np.array
    ([[2,1],[1,3]])\neigenvalues = np.linalg.eigvals(A)\nprint(eigenvalues)"}}
</tool_call>

<tool_response>
[1.38196601 3.61803399]
</tool_response>

The eigenvalues are approximately 1.382 and 3.618.
These are (5 +/- sqrt(5))/2, which are the golden ratio and its conjugate...
</think>
<answer>The eigenvalues are (5 +/- sqrt(5))/2</answer>
```

Listing 13.1: Tool-integrated reasoning format in QwQ

The RL training reward is computed on the final answer, but the model learns to use tools strategically because tool use improves the probability of reaching the correct answer.

13.6 Key Methods with Mathematical Foundations

13.6.1 Monte Carlo Tree Search for Reasoning

Monte Carlo Tree Search (MCTS) provides a principled framework for reasoning as tree search. In the AlphaProof [68] and related systems, MCTS is applied over reasoning steps rather than game moves.

State and Action Space

- **State** s_k: The partial reasoning chain $(q, r_1, r_2, \ldots, r_k)$ where r_i are reasoning steps
- **Action** a: The next reasoning step (a sentence or paragraph)
- **Terminal state**: A state containing a final answer
- **Reward**: $R(s_{\text{terminal}}) = r_{\text{acc}}$ (Eq. [eq:r1_accuracy_reward])

Value Function for Partial Solutions A value function $V(s_k)$ estimates the probability of reaching a correct answer from partial state s_k:

$$V(s_k) = P(\text{correct answer} \mid s_k) \approx \frac{1}{M} \sum_{m=1}^{M} R(\text{rollout}_m(s_k)) \tag{13.30}$$

where $\text{rollout}_m(s_k)$ is a Monte Carlo rollout from s_k to a terminal state using the current policy.

UCB Exploration Node selection uses the Upper Confidence Bound (UCB) formula adapted for reasoning:

$$\text{UCB}(s_k, a) = Q(s_k, a) + c_{\text{puct}} \cdot \pi_\theta(a \mid s_k) \cdot \frac{\sqrt{N(s_k)}}{1 + N(s_k, a)} \tag{13.31}$$

where:

- $Q(s_k, a) = \frac{1}{N(s_k,a)} \sum_{\text{visits}} V(s_{k+1})$ is the mean value of child states
- $\pi_\theta(a \mid s_k)$ is the policy prior (language model probability of step a)
- $N(s_k)$ is the visit count of state s_k
- $N(s_k, a)$ is the visit count of the (s_k, a) edge
- c_{puct} is the exploration constant

MCTS-Guided Training MCTS can be used to generate high-quality training data:

$$\mathcal{L}_{\text{MCTS}}(\theta) = -\sum_k \sum_a \pi_{\text{MCTS}}(a \mid s_k) \log \pi_\theta(a \mid s_k) \tag{13.32}$$

where $\pi_{\text{MCTS}}(a \mid s_k) \propto N(s_k, a)^{1/\tau}$ is the MCTS policy (visit count distribution with temperature τ).

13.6.2 Process Reward Models

Math-Shepherd: Automated PRM Training Math-Shepherd [365] proposes an automated method for training PRMs without human step-level annotations. The key insight is to use **outcome-based estimation**: a step s_k is labeled as correct if there exists a completion from s_k that reaches the correct answer.

Formally, for a partial solution $(s_1, \ldots, s_k)$:

$$\hat{r}_k = \mathbf{1}[\exists\, (s_{k+1}, \ldots, s_K) : \text{verify}(s_K, y^*) = 1] \tag{13.33}$$

In practice, this is estimated by sampling M completions from s_k and checking if any are correct:

$$\hat{r}_k \approx \mathbf{1}\left[\sum_{m=1}^{M} \text{verify}(\text{complete}_m(s_k), y^*) > 0\right] \tag{13.34}$$

The PRM is then trained with binary cross-entropy:

$$\mathcal{L}_{\text{PRM}}(\phi) = -\sum_{k=1}^{K} [\hat{r}_k \log r_\phi(s_k) + (1 - \hat{r}_k) \log(1 - r_\phi(s_k))] \tag{13.35}$$

PRM for Best-of-N Selection A primary application of PRMs is **best-of-N selection**: generate N candidate solutions and select the one with the highest PRM score:

$$y^* = \arg \max_{y \in \{y_1, \ldots, y_N\}} R_{\text{PRM}}(q, y) \tag{13.36}$$

This is more effective than majority voting (which uses ORM) because PRM can distinguish between solutions that reach the same answer via different quality reasoning paths.

13.6.3 Outcome Reward Models and Majority Voting

Majority Voting (Self-Consistency) The simplest form of test-time compute scaling is majority voting [367]: generate N solutions and return the most common answer:

$$y^* = \arg \max_a \sum_{i=1}^{N} \mathbf{1}[y_i = a] \tag{13.37}$$

Under the assumption that each solution is independently correct with probability $p > 0.5$, the probability that majority voting is correct is:

$$P(\text{majority correct}) = \sum_{k=\lceil N/2 \rceil}^{N} \binom{N}{k} p^k (1-p)^{N-k} \xrightarrow{N \to \infty} 1 \tag{13.38}$$

Weighted Majority Voting with ORM An ORM can improve majority voting by weighting votes by confidence:

$$y^* = \arg\max_a \sum_{i=1}^{N} R_{\text{ORM}}(q, y_i) \cdot \mathbf{1}[y_i = a] \tag{13.39}$$

13.6.4 Self-Play for Reasoning

Self-play methods generate training data by having the model play both the *generator* and *verifier* roles.

STaR: Self-Taught Reasoner STaR [418] bootstraps reasoning capabilities iteratively:

1. Generate reasoning chains for a problem set
2. Keep chains that lead to correct answers (rejection sampling)
3. Fine-tune on kept chains
4. Repeat with the improved model

The key insight is that the model can *rationalize* correct answers: even if it cannot solve a problem from scratch, it can generate a plausible reasoning chain given the answer, which can then be used as training data.

Self-Play RL In self-play RL for reasoning, the model generates both problems and solutions:

$$\mathcal{L}_{\text{self-play}}(\theta) = \mathbb{E}_{q \sim \pi_\theta^{\text{gen}}} \mathbb{E}_{y \sim \pi_\theta^{\text{solve}}(\cdot|q)} \left[r(y, y^*) \right] \tag{13.40}$$

where π_θ^{gen} generates problems and $\pi_\theta^{\text{solve}}$ solves them. The generator is rewarded for producing problems that are challenging but solvable.

13.6.5 Reinforcement Learning from Verifiable Rewards (RLVR)

RLVR [198] is a framework that uses **ground-truth verification** as the reward signal, applicable to any domain where correctness can be automatically checked.

Verifiable Domains

- **Mathematics**: Symbolic verification via SymPy, Lean, or Isabelle
- **Code**: Unit test execution
- **Formal logic**: Proof checking
- **Factual QA**: Database lookup
- **Games**: Win/loss outcome

RLVR Objective

$$\mathcal{L}_{\text{RLVR}}(\theta) = -\mathbb{E}_{(q,y^*)\sim\mathcal{D}} \mathbb{E}_{y \sim \pi_\theta(\cdot|q)} \left[\text{verify}(y, y^*) \right] + \beta \mathbb{D}_{\text{KL}}[\pi_\theta \,\|\, \pi_{\text{ref}}] \tag{13.41}$$

The key advantage of RLVR over RLHF is the **absence of reward model error**: since the reward is computed by a deterministic verifier rather than a learned model, there is no reward hacking against a flawed reward model. The only failure mode is if the model finds solutions that pass verification but are not genuinely correct (e.g., exploiting test case weaknesses in code evaluation).

RLVR for Code: Reward Hacking Challenges

In code generation, the verifier is a test suite. A model trained with RLVR can learn to:

- **Hardcode test outputs**: Return the expected output for each test input without implementing the actual algorithm
- **Exploit weak tests**: Pass all provided tests while failing on edge cases

Mitigations include: using large, diverse test suites; including adversarial test cases; using execution-based rewards that penalize hardcoding (e.g., checking that the solution runs in $O(n \log n)$ time).

13.6.6 Journey Learning

Journey Learning [298] proposes training on the **full reasoning trajectory**, including failed attempts and corrections, rather than only successful final solutions.

Motivation Standard rejection sampling discards failed attempts. But failed attempts contain valuable information:

- Which approaches don't work (negative examples)
- How to recognize and recover from errors (correction patterns)
- The structure of the problem space (exploration data)

Journey Learning Objective Given a trajectory $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$ that may include backtracking:

$$\mathcal{L}_{\text{journey}}(\theta) = -\sum_{t=0}^{T} w_t \log \pi_\theta(a_t \mid s_t) \tag{13.42}$$

where the weights w_t are designed to emphasize:

- Steps that lead to eventual success ($w_t > 1$)
- Correction steps after errors ($w_t > 1$)
- Steps in failed branches ($w_t < 1$, but > 0)

13.6.7 Quiet-STaR: Reasoning at Every Token

Quiet-STaR [419] extends the reasoning paradigm to **every token position**: rather than generating a reasoning chain only before the final answer, the model generates a "thought" at every token position.

Formulation For each token position t, the model generates a hidden thought z_t before predicting the next token x_{t+1}:

$$P(x_{t+1} \mid x_{\le t}) = \mathbb{E}_{z_t \sim \pi_\theta(\cdot \mid x_{\le t})} \left[\pi_\theta(x_{t+1} \mid x_{\le t}, z_t) \right] \tag{13.43}$$

In practice, this is approximated by mixing the predictions with and without the thought:

$$P(x_{t+1} \mid x_{\le t}) = \alpha \cdot \pi_\theta(x_{t+1} \mid x_{\le t}, z_t) + (1 - \alpha) \cdot \pi_\theta(x_{t+1} \mid x_{\le t}) \tag{13.44}$$

Training with REINFORCE Since the thought z_t is a discrete latent variable, the gradient is estimated using REINFORCE:

$$\nabla_\theta \mathcal{L}_{\text{QS}} = \mathbb{E}_{z_t} \left[\nabla_\theta \log \pi_\theta(z_t \mid x_{\le t}) \cdot (\log P(x_{t+1} \mid x_{\le t}, z_t) - b_t) \right] \tag{13.45}$$

where b_t is a baseline (e.g., the no-thought prediction $\log \pi_\theta(x_{t+1} \mid x_{\le t})$).

Computational Cost of Quiet-STaR

Quiet-STaR increases inference cost by a factor of $L_z + 1$ where L_z is the thought length, applied at *every* token position. For a sequence of length T with thoughts of length $L_z = 8$, this is a $9\times$ increase in compute. This makes Quiet-STaR impractical for long sequences without significant engineering optimizations (e.g., speculative decoding for thoughts, caching).

13.7 Scaling Laws for Reasoning

Recent work [335, 388] has established that test-time compute scales predictably with reasoning performance, extending the classical scaling laws [177] into the inference regime.

13.7.1 Training Compute vs. Test-Time Compute Tradeoff

The fundamental scaling question for reasoning models is: **given a fixed total compute budget $C_{\text{total}} = C_{\text{train}} + N \cdot C_{\text{test}}$ (where N is the number of queries), how should compute be allocated?**

Let $\mathcal{A}(C_{\text{train}}, C_{\text{test}})$ denote the accuracy of a model trained with C_{train} FLOPs and given C_{test} inference FLOPs per query. Empirically:

$$\mathcal{A}(C_{\text{train}}, C_{\text{test}}) \approx 1 - \exp\left(-a \cdot C_{\text{train}}^{\alpha} \cdot C_{\text{test}}^{\beta}\right) \tag{13.46}$$

for constants $a, \alpha, \beta > 0$. The optimal allocation for a fixed total budget C_{total} satisfies the condition that marginal return per FLOP is equalized between training and inference:

$$\frac{\partial \mathcal{A}}{\partial C_{\text{train}}} = \frac{1}{N} \cdot \frac{\partial \mathcal{A}}{\partial C_{\text{test}}} \tag{13.47}$$

Intuitively: one FLOP of training benefits all N queries, while one FLOP of test-time benefits only one query. At the optimum, the per-query marginal value of test-time compute is N times larger than training compute (because training is amortized). Applying this to Eq. [eq:reasoning_scaling_law] gives the optimal training compute fraction:

$$\frac{C_{\text{train}}^{*}}{C_{\text{total}}} = \frac{\alpha}{\alpha + \beta} \tag{13.48}$$

For the specific budget structure $C_{\text{total}} = C_{\text{train}} + N \cdot C_{\text{test}}$, this fraction is independent of N under the multiplicative accuracy model. However, in practice α and β are problem-dependent: for high-volume deployments (large N), even small improvements in the base model dominate, favoring training investment. For low-volume, high-stakes queries (small N), test-time compute is more cost-effective.

13.7.2 When to Invest in Longer Chains vs. Better Base Models

Reasoning Chain Length vs. Model Capacity

The optimal reasoning chain length L^* for a model of capacity C on a problem of difficulty D satisfies:

$$L^* \propto \frac{D}{C^{\gamma}} \tag{13.49}$$

for some $\gamma > 0$. This implies:

- **Hard problems** require longer chains regardless of model size
- **Larger models** require shorter chains for the same problem difficulty
- **Diminishing returns**: Beyond L^*, additional tokens provide no benefit and may hurt (overthinking)

The "overthinking" phenomenon—where models with very long reasoning chains perform *worse* than those with moderate chains—has been empirically observed and is attributed to:

- Accumulation of errors in long chains (error propagation)
- Distraction from the main solution path
- Overconfidence in incorrect intermediate conclusions

13.7.3 Optimal Token Budget Allocation

For a model with a fixed token budget B, the allocation between "thinking" tokens T_{think} and "answering" tokens T_{answer} should satisfy:

$$T^*_{\text{think}} = \arg\max_{T} \mathcal{A}(T, B - T) \tag{13.50}$$

Empirically, the optimal split is problem-dependent:

- **Simple problems**: $T^*_{\text{think}}/B \approx 0.3$ (30% thinking)
- **Hard problems**: $T^*_{\text{think}}/B \approx 0.8$ (80% thinking)
- **Very hard problems**: $T^*_{\text{think}}/B \approx 0.95$ (95% thinking, minimal answer)

This motivates **adaptive thinking budgets**: allocating more tokens to harder problems, which can be estimated by the model's uncertainty on initial solution attempts.

13.8 Comparison of Reasoning Models

Table 13.3: Comparison of training methodologies for reasoning models.

Method	PRM	ORM	MCTS	Distill	Tool	Open
OpenAI o1/o3	✓	✓	Unknown	–	✓	×
DeepSeek-R1	×	✓	×	✓	×	✓
QwQ / Qwen3	Partial	✓	×	×	✓	✓
AlphaProof	✓	✓	✓	–	✓	×
Math-Shepherd	✓	✓	×	–	×	✓
STaR / Quiet-STaR	×	✓	×	–	×	✓

13.9 Inference-Time Compute Scaling Laws

Training-time scaling laws (Chinchilla [140]) are well-understood: model performance improves predictably with compute, data, and parameters. **Inference-time compute scaling** is the new frontier—the empirical finding that allocating more computation *at test time* (longer reasoning chains, more search, iterative refinement) can solve problems that larger base models cannot.

The Core Finding: Test-Time Compute as a Scaling Axis

Snell et al. (2024) [335] established that test-time compute can be more effective than scaling model parameters for hard reasoning tasks:

- Performance improves **log-linearly** with inference tokens: acc $\approx a \cdot \log(\text{tokens}) + b$.
- There exist **task-specific optimal token budgets** beyond which returns diminish sharply.
- A smaller model with $k\times$ more inference compute can match a larger model trained with $k\times$ more parameters—at lower serving cost for rare hard queries.

Empirical landmark: **o3** achieved **45.1% on ARC-AGI** (a benchmark designed to resist pattern matching) by scaling inference-time search, far exceeding what model size alone could achieve.

13.9.1 The Overthinking Problem

Inference-time scaling introduces a critical failure mode: **overthinking**. Reasoning models trained to generate long chains of thought continue generating tokens even when the answer is obvious, wasting compute and sometimes degrading accuracy by introducing spurious reasoning steps.

Overthinking: When More Tokens Hurt

Reasoning models exhibit a characteristic **U-shaped accuracy curve** as a function of token budget on easy problems:

1. Short chains: insufficient reasoning, low accuracy.
2. Optimal chains: correct reasoning, peak accuracy.
3. Excessively long chains: the model second-guesses correct answers, introduces errors, accuracy *decreases*.

This means that a fixed "think as long as possible" policy is suboptimal. Models need to learn **when to stop thinking**.

13.9.2 Efficient Reasoning Techniques

Several approaches address the compute-quality trade-off in inference-time scaling:

- **Sketch-of-Thought**: Generate a compressed reasoning sketch before the full chain. Achieves **84% token reduction** with minimal accuracy loss by separating planning from execution in the reasoning trace.
- **Budget-aware prompting**: Explicitly instruct the model with a token budget ("solve this in under 200 tokens"). Halves inference costs on benchmarks where problems have known difficulty.
- **Shorter chains boost accuracy**: On some tasks, enforcing shorter reasoning chains improves accuracy by **34.5%**—the model is forced to be more direct and avoids overthinking spirals.
- **Adaptive depth**: OpenAI's `reasoning_effort` parameter (low/medium/high) routes queries to different token budgets based on estimated difficulty, providing a practical API for compute-quality trade-offs.

13.9.3 Connection to RL: Inference Compute as MDP Horizon

Inference-Time Compute in the MDP Framework

Inference-time compute scaling has a natural interpretation in the RL framework used throughout this book:

- Each reasoning token is a **step in the MDP**.
- Allocating more inference compute = **longer horizon** with the same reward function.
- The overthinking problem = **reward hacking** on a proxy (token count) rather than task success.
- Efficient reasoning = learning an **optimal stopping policy** within the MDP.

This framing suggests that inference-time scaling laws are not merely empirical curiosities—they

reflect fundamental properties of the RL optimization landscape. Models trained with RL on reasoning tasks are implicitly learning both *what to think* and *how long to think*.

13.9.4 Latent Reasoning: Is the Chain the Computation?

A growing body of evidence challenges the assumption that chain-of-thought *is* the reasoning:

- **Reasoning Beyond Chain-of-Thought** [168]: Using Sparse Autoencoders to probe internal representations, this work identifies reasoning-specific features that activate *before* the corresponding tokens are generated—suggesting the model "knows the answer" before writing the chain.
- **Thinking to Recall** [115] (Google, COLM 2026): Demonstrates that generating a reasoning trace unlocks parametric knowledge that is otherwise effectively unreachable via direct prompting—but the trace itself may be a *scaffold* rather than the computation.

CoT as Communication Protocol

The emerging view: chain-of-thought may function as a *communication protocol* between the model's internal reasoning circuits and its output layer—a way to "route" through latent states that contain the answer—rather than being the algorithm itself. This has practical implications: (1) verifying reasoning quality cannot rely solely on inspecting the chain, since the model may reason correctly with a misleading or post-hoc trace; (2) future architectures may perform reasoning entirely in latent space, emitting only conclusions.

13.10 On-Policy Self-Distillation for Reasoning

The methods in this chapter—GRPO, process rewards, MCTS—share a fundamental limitation: **sparse feedback**. Reinforcement learning provides a fixed number of bits per episode regardless of sequence length. A 2000-token chain-of-thought that arrives at the wrong answer receives a single scalar penalty; the model does not learn *which tokens* contributed to the failure. On-Policy Self-Distillation (OPSD) [430] addresses this by providing **dense, token-level supervision** while retaining the on-policy property that makes RL effective.

On-Policy Self-Distillation: Core Idea

A single LLM plays two roles simultaneously:

- **Teacher**: the same model conditioned on *privileged information* (e.g., a verified solution or reasoning trace appended to the prompt).
- **Student**: the model with only the original question as input, generating its own rollouts.

Training minimizes the per-token divergence between the teacher's and student's distributions, evaluated on the *student's own trajectories*. No separate, larger teacher model is required.

13.10.1 The Training Objective

Let π_θ denote the student policy (the model given only the question) and π_θ^+ denote the teacher policy (the same model conditioned on privileged context such as a verified answer). For a trajectory $x = (x_1, \ldots, x_T)$ sampled from the student, the OPSD loss minimizes the per-token reverse KL divergence:

$$\mathcal{L}_{\text{OPSD}}(\theta) = \mathbb{E}_{x\sim\pi_\theta}\left[\sum_{t=1}^{T} \text{KL}\Big(\pi_\theta(\cdot \mid x_{<t}) \,\|\, \pi_\theta^+(\cdot \mid x_{<t})\Big)\right]$$

In practice, this simplifies to the difference in log-probabilities evaluated on the sampled tokens:

$$\mathcal{L}_{\text{OPSD}}(\theta) \approx \mathbb{E}_{x\sim\pi_\theta}\left[\sum_{t=1}^{T} \log \pi_\theta(x_t \mid x_{<t}) - \log \pi_\theta^+(x_t \mid x_{<t})\right]$$

The teacher's log-probabilities require only a single forward pass (no gradient computation), making the approach significantly cheaper than running a separate reward model or performing full RL rollouts with delayed rewards.

13.10.2 Why Dense Supervision Matters

The information-theoretic argument is compelling: RL teaches $O(1)$ bits per episode (the scalar reward), while token-level distillation teaches $O(T)$ bits per episode. This translates directly into training efficiency:

- **Qwen3 results**: The Qwen3 technical report reports that on-policy distillation achieves 74.4% on AIME'24 at **one-tenth the GPU-hours** of their RL pipeline (1,800 vs. 17,920 GPU-hours), while exceeding RL's 67.6%.
- **Step efficiency**: Empirical comparisons show OPSD reaching equivalent reasoning performance in 7–10× fewer gradient steps than GRPO, corresponding to 50–100× compute savings when accounting for shorter required context lengths and smaller batch sizes.
- **Data reuse**: Unlike RL, which memorizes final answers when trained on repeated prompts, OPSD learns to approximate the teacher's full distribution via reverse KL, enabling effective multi-epoch training on small prompt sets.

13.10.3 Failure Modes and Mitigations

Privilege-Induced Style Drift

When the teacher is conditioned on a verified solution, it tends to produce shorter, more direct outputs—it already "knows" the answer. The resulting distributional gap between teacher and student concentrates on *style tokens* (brevity, directness) rather than *task-bearing tokens* (correct reasoning steps). This causes the student's response length to shrink without improving accuracy.

RLCSD [282] addresses this pathology through a **contrastive** formulation: it computes the teacher-student gap under both a correct hint and an incorrect hint, then subtracts the latter to cancel out the style component that conditioning on *any* hint induces. The residual signal concentrates on task-relevant tokens.

EGRSD [181] takes a complementary approach: it introduces an **entropy-guided confidence gate** that down-weights token positions where the teacher's predictive distribution is high-entropy (uncertain), focusing the gradient on positions where the teacher is confident and the student diverges.

13.10.4 Connection to RL and the Training Pipeline

OPSD occupies a specific niche in the post-training pipeline:

Figure 13.6: OPSD bridges SFT and RL: it combines on-policy sampling (avoiding distribution mismatch) with dense token-level supervision (avoiding reward sparsity).

The practical pipeline emerging in frontier labs is: (1) SFT on teacher-generated reasoning traces to initialize the policy within the correct distribution; (2) OPSD to cheaply recover or refine reasoning capabilities with dense supervision; (3) RL for the final push on hard problems where OPSD's teacher signal becomes unreliable (the teacher itself cannot solve them).

When to Use OPSD vs. RL

- **Use OPSD**: When the teacher (or the model itself with privileged context) reliably solves the target tasks; when compute budget is limited; when you need to recover reasoning behavior after domain-specific fine-tuning; for continual learning where RL's sparsity causes

catastrophic forgetting.

- **Use RL**: When pushing beyond the teacher's capability ceiling; when exploring novel reasoning strategies that don't exist in any teacher distribution; when verifiable rewards are cheap (code execution, math verification).
- **Use both**: OPSD first to cheaply reach the teacher's level, then RL to push beyond it.

13.11 Summary and Open Problems

The field of RL for reasoning models has advanced remarkably rapidly. Several key lessons have emerged:

1. **Verifiable rewards are sufficient**: For domains with ground-truth verification (math, code), outcome-only rewards are sufficient for RL to discover sophisticated reasoning strategies, without requiring process reward models.
2. **Test-time compute is a new axis**: Reasoning models introduce a new dimension of scaling—inference compute—that is roughly substitutable with training compute for hard reasoning tasks.
3. **Distillation is highly effective**: Large reasoning models can transfer their capabilities to much smaller models via supervised fine-tuning on generated chains, often outperforming direct RL training of small models. On-policy self-distillation further closes the gap between distillation and RL at a fraction of the compute.
4. **Emergent meta-cognition**: RL training on reasoning tasks produces emergent self-correction and verification behaviors that were not explicitly trained.
5. **Dense supervision beats sparse rewards**: On-policy self-distillation achieves RL-level reasoning at 10–100× lower compute by providing token-level feedback rather than episode-level rewards.

Open Problems in RL for Reasoning

Several fundamental questions remain open:

- **Generalization**: Do reasoning capabilities trained on math/code transfer to other domains (scientific reasoning, planning, social reasoning)?
- **Faithfulness**: Are the generated reasoning chains causally responsible for the final answer, or are they post-hoc rationalizations? Latent reasoning research suggests the chain may be a communication protocol rather than the computation itself.
- **Optimal search**: What is the optimal search strategy during inference—beam search, MCTS, or something else?
- **Reward design**: For domains without ground-truth verifiers, how can we design reliable reward signals for reasoning?
- **Overthinking**: How can models learn to allocate the *right* amount of thinking—neither too little nor too much? Inference-time scaling laws show diminishing and eventually negative returns beyond task-specific optimal budgets.
- **Compositional reasoning**: Can RL-trained reasoning models solve problems that require composing multiple distinct reasoning skills?
- **Latent vs. explicit reasoning**: Can future architectures reason entirely in continuous latent space, emitting only conclusions—and if so, how do we verify their reasoning?